SET 4

1. Load Balancing across 2 servers Using Packet Tracer

Aim:

To design and configure a network using using Cisco packet Tracer to demonstrate basic load distribution across two servers.

Procedure

1. Open Cisco Packet Tracer.
2. Add:
   * 1 Router
   * 1 Switch
   * 2 Servers
   * 2 PCs
3. Connect the PCs and Servers to the Switch using copper straight-Through Cables.
4. Connect the switch to the router
5. Configure the IP addresses:

Device	IP Address	Subnet Mask
Router	192.168.1.1	255.255.255.0
Server1	192.168.1.10	255.255.255.0

Server 2	192.168.1.20	255.255.255.0
PC1	192.168.1.30	255.255.255.0
PC2	192.168.1.40	255.255.255.0

6. Enable the required service, such as HTTP, on both servers.

7. Configure the network gateway as 192.168.1.1.

8. Test connectivity using the ping command.

9. Send requests from the client PCs to the two servers.

10. Demonstrate basic load distribution by directing different client requests to server 1 and server 2.

Sample Output

PC1 > Ping 192.168.1.10

Reply from 192.168.1.10 : bytes=32 time<1ms

Reply from 192.168.1.10 : bytes = 32 time <1ms

PC2 >Ping 192.168.1.20

Reply from 192.168.1.20 : bytes = 32 time<1ms

Reply from 192.168.1.20 : bytes =32 time <1ms

Result

The network was successfully designed and configured in Packet Tracer, and basic load distribution across two servers was demonstrated successfully.

2. Port Security Using Packet Tracer

Aim

To configure Port Security on a switch to allow only specific MAC addresses and block unauthorized devices.

Procedure

1. Open Cisco Packet Tracer
2. Add:
   * 1 Switch
   * 2 PCs
3. Connect both PCs to the switch
4. Check the MAC address of the authorized PC
5. Open the switch CLI
6. Enter the following commands:

```
Switch>enable
Switch# configure terminal
Switch(config)# switchport mode access
Switch(config)# switchport port-security
```

7. Connect the authorized PC to Fa0/1.
8. Generate traffic using ping.

9. Replace the authorized PC with another unauthorized PC.

10. Generate traffic again

11. The switch detects the unauthorized MAC address and places the port into a Security-violation state.

Sample output

For checking Port Security:

```
Switch # Show port-security interface
fast Ethernet 0/1
Port Security      : Enabled
Port Status        : Secure-up
Maximum MAC Addresses : 1
Total MAC Addresses : 1
Security Violation Count : 0
```

Result:

Port Security was successfully configured and the unauthorized device was blocked from accessing the switch port.

3. Traceroute Packet Analysis Using wireshark.

Aim

To capture and analyze packets generated during a Traceroute operation using Wireshark, particularly ICMP TTL Exceeded Packets and study their header fields and Payload

Procedure

1. Install and open wireshark
2. Select the active network interface
3. Start packet capture.
4. Open Command Prompt / Terminal

5.Run

Windows:

```
tracert google.com
```

Linux:

```
traceroute google.com
```

6. Observe the packets captured by wireshark.
7. Apply the display filter.
   icmp
8. Select an ICMP Time-to-live exceeded Packet
9. Analyze
10. Stop the capture and record the observation.

Sample Output

Traceroute:

Tracing route to google.com

1	<1 ms	<1ms	<1ms	192.168.1.1
2	5ms	6ms	5ms	10.101.1
3	15ms	14ms	16ms	...

Wireshark Observation

Protocol : ICMP

ICMP Type : 11

ICMP Code : 0

Message : Time-to-live exceeded

TTL : 64

Source IP : 192.168.1.1

Destination IP: Client IP

Important ICMP values

Field	Value
ICMP	11
ICMP Code	0
Meaning	Time-to live exceeded
Protocol	ICMP

Result

Traceroute packets were successfully captured using Wireshark, and the ICMP TTL Exceeded Packet headers and payload were analyzed successfully.

4. Parity Bit Generation and Error checking Using C

Aim:

To implement parity bit generation and checking using a C Program for detecting errors in transmitted data.

Procedure

1. Start the C program.
2. Read the binary data from the user.
3. Count the number of 1s in the data.
4. For even parity, generate:
   * Parity bit 0 if number of 1s is even
   * Parity bit 1 if number of 1s is odd.
5. Append the parity bit to the data
6. Read the received data.
7. Count the number of 1s again
8. Check whether the total number of 1s is even
9. If even -> No error detected
10. If odd -> Error detected.

C program

```
#include <stdio.h>
# include <string.h>
int main()
{
 char data [100];
  int i, count =0;
 Print ("Enter binary data:");
  Scanf ("%s", data);
 for(i=0; i < strlen (data):i++)
 { if (data [i]== '1')
     count ++;
  }
  if (count % 2== 0)
      Printf ("Even parity bit:= 0\n");
  else
      Print ("Even Parity bit =1\n");
  Print ("Parity checking Completed.\n");
  return 0;
  }
```

sample output

Input:

Enter binary data: 1011001

Number of 1s = 4

Even parity bit = 0

Parity checking completed

Result

The C program was successfully implemented to generate and check the Parity bit and parity-based error detection was demonstrated successfully.